

CPU Algorithm Design

Exercise 2

- Submit electronically (MOODLE) as one archive named after the group members.
- Archive should encompass a PDF and your code.
- Include names on the top of the sheets.
- A maximum of two students is allowed to work jointly on the exercises.

Accessing Server

First, copy the code to the server

```
scp ex02-code.tar.gz <uni-id>@asc-headnode.ziti.uni-heidelberg.de:<path>
```

You can also do this with gitlab or github by creating your own repository and upload the code, then pull the code on the server. Setting path to /media/oldhome/[first letter of your first name + your last name] is recommended.

Start your favourite ssh client and connect to the server using

```
ssh <uni-id>@asc-headnode.ziti.uni-heidelberg.de
```

Then use your university password to login. After login, use the following command to go to the working directory.(you could do everything in the default login directory, but the resources(disk memory mainly) on the login node is limited).

```
cd /media/oldhome/[first letter of your first name + your last name]  
tar -xvzf ex02.tar.gz
```

Then you can start working.

Submit Code to Run on The Server

You could build and run your code on the login node for testing purpose. However, since we have many different hardware and for this course performance is the key, results generated with login node usually is not comprehensive.

Clearly, you need some third party library to build your code. To add libraries into your private session on the server, you can use

```
spack load [name of the package]
```

to load the packages. We provide you a script where all the dependency is included, just type

```
source load_env_CPUAD.sh
```

this will load all the dependencies and set the correct environment variable for you. You need to repeat this everytime you login to a new session.

After loading all the dependencies, it's time to build your code. We use cmake to generate Makefiles for the code.

```
mkdir build  
cd build  
cmake ..
```

this will generate the Makefile in the build directory. Then you can call

```
make
```

to generate the executable. If you only want to generate a specific executable you can add argument to **make**. For example

```
make benchmarkSIMDreduce
```

will build the executable for `SIMD_reduce_bench.cpp`. This is helpful when you only have partial working code. The correspondence of the argument and cpp file can be find in `CMakeLists.txt`, where the first argument of `add_executable` is the argument for `make` and the second argument is the cpp file you're building.

Now we have a executable, how to execute it on the server. On our server, we use slurm to manage our jobs. What you need to do is to create a script and submit the script using

```
sbatch your_script
```

Then the server will print a job id and schedule the job.

```
squeue
```

This will print all the jobs in queue and execution status.

```
scancel jobid
```

This will cancel the run for `jobid`. We already provide you with a initial script to submit the the code `ex02SlurmScript.sh`.

```
#SBATCH -w asc-rome00
```

We advise you to change `00` to `your group id % 4` such that the workload is evenly distributed.

```
#SBATCH -o ex02out.txt
```

You can change the name and the location of your output file by changing `ex02out.txt`. but make sure that the path exist.

```
./benchmarkSIMDreduce
```

Lastly, you can change the executable you want to run by changing this.

If you want to learn more detail about this workflow, check out the ‘Getting start’ section of our group [wiki](#).

File List

1. `plotRes.py`: plotting script
2. `main_SIMD_reduce.cpp` and `main_SIMD_transform.cpp`: example of how to run the code with small `N` for debug purpose.
3. `SIMD_reduce.hpp` and `SIMD_transform.hpp`: reduce and transform using normal containers. You shouldn’t change these files.
4. `SIMD_reduce_view.hpp` and `SIMD_transform_view.hpp`: reduce and transform using `views(std::array)`. You should complete these files.
5. `SIMD_reduce_bench.cpp` and `SIMD_transform_bench.cpp`: benchmarking code for reduce and transform using containers. You should submit the executable built from these to the server.

6. `SIMD_reduceV_bench.cpp` and `SIMD_transformV_bench.cpp`: benchmarking code for reduce and transform using `views(std::array)`. You should submit the executable built from these to the server.
7. `load_env_CPUAD.sh`: script to load the dependencies to your session. You shouldn't change this.
8. `ex02SlurmScript.sh`: script to submit your executable to the server. Base on your need, you should change the `rome00`, the name of the executable and the name of the output file.
9. `utilities.hpp` and `benchmark.hpp`: utility functions for the code and benchmarking. You shouldn't change these.
10. `CMakeLists.txt`: file for cmake to generate the makefile. You shouldn't change these.(for debug purpose, you can change the `-O3` to `-g` at 10th line, but make sure the result is generated with `-O3` flag)

Suggestions

1. Pay attention to the working directory in which you run your command.
2. You can change `Nl` and `Nh` to smaller values for faster prototyping, but make sure the result is generate using the original values.(usually `Nh=Nl+1` and this means your container is of size `Nl`).

1 Performance of reduce and transform (10 points)

Build both `SIMD_transform_bench.cpp` and `SIMD_reduce_bench.cpp`. After compiling, submit the jobs to the `rome` server for execution. Once the benchmarks complete, use the `plotRes.py` script to visualize the results. Analyze the resulting plots. What trends or patterns do you observe in the performance data?

2 Adapting reduce and transform (30 points)

1. Complete the `SIMD_reduce_view.hpp` except for three `xsimd` function. In `SIMD_reduce.cpp`, the data is loaded from the container which is in memory. However, the containers are initialized with uniform values. Now, in `SIMD_reduce_view.hpp`, replace the container with `std::views::repeat` so that the code is not memory bound anymore.
2. Complete the `SIMD_transform_view.hpp`. In `SIMD_transform.cpp`, the data is loaded from the container which is in memory. However, the containers are initialized with values with specific pattern(`a, a+1, a+2...a+n`). Now, in `SIMD_transform_view.hpp`, replace the container with `std::ranges::views::iota` so that the code is not memory bound anymore. For transform you also need to store the result into an container. As we're computing $W = a*V + W$, use `views::iota`

for `V`, and a container(`std::vector`) of size 256 for `W`. For `W`, use modulo indexing (`i%n`) for reading and writing. You could wrap your container with a `view` for those functions that doesn't access element with index. Or have an extra counter and perform iterator arithmetic.

3. Repeat the analysis in problem 1, but with `SIMD_transformV_bench.cpp` and `SIMD_reduceV_bench.cpp`. Now, what trends or patterns do you observe in the performance data?

Remark

When adapting the functions, please try to change as less code as possible. All the function inside of the same class perform the same task, copying the same code ten times will satisfy the functionality requirements but it's not what we are looking for. For example, if the original code uses `transform` with `std::execution::unseq`, then the adapted version should also uses `transform` with `std::execution::unseq`. Also make sure that the data type is still `Real`.

3 Adapting xsimd reduce and transform. (30+5 points)

Complete the `benchReduceSimdXHorizontal`, `benchReduceSimdXVertical`, `benchReduceSimdXAlign` functions in `SIMD_reduce_view.hpp`. `benchReduceSimdXHorizontal` and `benchReduceSimdXVertical` are not aligned. In problem 2 we want to avoid loading data from memory. However, this is not possible for `xsimd` since it needs to take the memory address to create the `batch`. The return value of `std::views::repeat` is a temporary variable, and taking the address of a temporary variable is dangerous. Now, since we know the values are repeated, we can create a `std::array` of size `3*simd_width`. This way, we can load the data repeatedly from the same section of the `array` without worrying to have an invalid address. Add the result you get to the previous plot.

Explain the difference between the Horizontal version and Vertical version and performance indication.(5 points)

Complete the `benchXsimdTransformAligned`, `benchXsimdTransform` functions in `SIMD_transform_view.hpp`. You should do the same thing just as in `reduce`. The difference is in `reduce` all the element in the array is the same, but here you have increasing values. You should initialize the part of the array where you're trying to access to `{1,2,3....simd_width}` and `{2,3,4....simd_width+1}` for `V` and `W` respectively. this means in the loop body you're always doing the same computation. And this is different with other functions in `transform` class, so you should get different result comparing with other functions.

Read the remark section before you start this problem.

Remark

1. Why not just using a `Real`?

The `batch::load_aligned` function loads multiple values at the same time. For example, `batch::load_aligned(&a)` means loading `a[0]`, `a[1]`, `a[2]` ... `a[simd_width-1]`. If `a` is a `Real`, then this operation will very likely to create a segfault since you have no control on what's at the memory location `(&a) + 1`.

2. Why 3 in `3*simd_width`?

As the function names indicates, there's a difference in loading from aligned location and unaligned location. Then, what does 'aligned' means? It means that the starting memory location of the data structure is aligned with the cacheline, this means when you're loading some data with size smaller than the cacheline size, you only need to touch one cacheline.

By default, when you allocate an array of some primitive type, you are guaranteed that the addresses of all elements are a multiple of their size, which ensures that they only span a single cache line. For example, you are guaranteed the address of the first and every other element of an int array is a multiple of 4 bytes (`sizeof int`). Sometimes you need to ensure that this minimum alignment is higher. For example, many SIMD applications read and write data in blocks of 32 bytes, and it is crucial for performance that these 32 bytes belong to the same cache line. (<https://en.algorithmica.org/hpc/cpu-cache/alignment/>)

Now, for `std::array` of `Real`, it is guaranteed to have the starting memory location to be a multiple of 4. However, cache line alignment requires it to be a multiple of 64. Here's the situation: we have an array starting with memory location which is a multiple of 4, we are loading 32 byte data, we want these 32 byte to be inside the same cache line (64 aligned). The solution is that you find the first memory address that is a multiple of 32 and load the next 32 byte. and you're guaranteed to find such data block within a 64 bytes starting from any memory location. Hence, you only need `2*simd_width` to guarantee this. However, to create unaligned, it is a little bit more difficult. It requires a `std::array` that of size `3*simd_width` because you need to guarantee that inside the 32 byte memory you're loading, you touch two different 64 byte chunk. this is not possible if your `2*simd_width` array's starting memory location is a multiple of 64. Hence the 3. And we want to make it uniform for both of the cases, so `3*simd_width` is the final decision.

4 Precision (10 points)

For reduction, why is the final result of the reduction different? Which is the correct result, if none of them are correct, how to get the correct result? (You can retrieve the final result by uncomment the `std::cout << test.get_log();` from the benchmark cpp file)

Submission

Submit `SIMD_reduce_view.hpp` and `SIMD_transform_view.hpp`, the plots and the text answers.